

Lecture 14: Introduction to Deep Learning Theory

Tao LIN

SoE, Westlake University

December 23, 2025

- 1 Introduction and Motivation
- 2 Universal Approximation Theory
- 3 Capacity and Complexity
- 4 Optimization Theory and Loss Landscapes
 - Gradient Descent Dynamics
 - Implicit Regularization
- 5 Generalization Theory
- 6 Modern Theoretical Frameworks
 - Neural Tangent Kernel (NTK)
 - Mean-Field and Information Theory
- 7 Advanced Topics and Frontiers
 - Robustness and Adversarial Examples
 - Theory of Generative Models
- 8 Summary and Conclusion

Table of Contents

- 1 Introduction and Motivation
- 2 Universal Approximation Theory
- 3 Capacity and Complexity
- 4 Optimization Theory and Loss Landscapes
- 5 Generalization Theory
- 6 Modern Theoretical Frameworks
- 7 Advanced Topics and Frontiers
- 8 Summary and Conclusion

The Black Box Mystery

- **Practice:** We can train models with billions of parameters that work incredibly well.

The Black Box Mystery

- **Practice:** We can train models with billions of parameters that work incredibly well.
- **Theory:** Traditional machine learning tools (VC dimension, Rademacher complexity) often predict that these models shouldn't work at all (overfitting).

The Black Box Mystery

- **Practice:** We can train models with billions of parameters that work incredibly well.
- **Theory:** Traditional machine learning tools (VC dimension, Rademacher complexity) often predict that these models shouldn't work at all (overfitting).
- **The Gap:** We have a “black box” that works, but we don't fully understand *why* it works so well.

Why Study Deep Learning Theory?

Empirical Success:

Why Study Deep Learning Theory?

Empirical Success:

- **Perception:** ImageNet, Object Detection

Why Study Deep Learning Theory?

Empirical Success:

- **Perception:** ImageNet, Object Detection
- **Reasoning:** GPT-4, Claude, LLMs

Why Study Deep Learning Theory?

Empirical Success:

- **Perception:** ImageNet, Object Detection
- **Reasoning:** GPT-4, Claude, LLMs
- **Science:** AlphaFold, Quantum Chem

Why Study Deep Learning Theory?

Empirical Success:

- **Perception:** ImageNet, Object Detection
- **Reasoning:** GPT-4, Claude, LLMs
- **Science:** AlphaFold, Quantum Chem
- **Control:** Robotics, AlphaGo

Why Study Deep Learning Theory?

Empirical Success:

- **Perception:** ImageNet, Object Detection
- **Reasoning:** GPT-4, Claude, LLMs
- **Science:** AlphaFold, Quantum Chem
- **Control:** Robotics, AlphaGo

The “Missing” Manual

- Why do overparameterized networks generalize?
- What makes optimization work despite non-convexity?
- How do networks learn representations?

Why Study Deep Learning Theory?

Empirical Success:

- **Perception:** ImageNet, Object Detection
- **Reasoning:** GPT-4, Claude, LLMs
- **Science:** AlphaFold, Quantum Chem
- **Control:** Robotics, AlphaGo

The “Missing” Manual

- Why do overparameterized networks generalize?
- What makes optimization work despite non-convexity?
- How do networks learn representations?

Goal: Moving from “Alchemy” to “Chemistry”

The Three Pillars of Deep Learning Theory

Expressivity
(Approximation)

Can NNs represent
the target function?

Optimization
(Dynamics)

Can we find the
optimal parameters?

Generalization
(Stability)

Will it work on
unseen data?

The Three Pillars of Deep Learning Theory

Width/Depth?

Expressivity
(Approximation)

Can NNs represent
the target function?

Optimization
(Dynamics)

Can we find the
optimal parameters?

Generalization
(Stability)

Will it work on
unseen data?

The Three Pillars of Deep Learning Theory

Width/Depth?

Expressivity
(Approximation)

Can NNs represent
the target function?

Landscape?

Optimization
(Dynamics)

Can we find the
optimal parameters?

Generalization
(Stability)

Will it work on
unseen data?

The Three Pillars of Deep Learning Theory

Width/Depth?

Expressivity
(Approximation)

Can NNs represent
the target function?

Landscape?

Optimization
(Dynamics)

Can we find the
optimal parameters?

Double Descent?

Generalization
(Stability)

Will it work on
unseen data?

The Three Pillars of Deep Learning Theory

Width/Depth?

Expressivity
(Approximation)

Can NNs represent
the target function?

Landscape?

Optimization
(Dynamics)

Can we find the
optimal parameters?

Double Descent?

Generalization
(Stability)

Will it work on
unseen data?

A complete theory must address the **interplay** between these three.

Course Roadmap

Course Roadmap

- 1 **Foundations:** Approximation theory, expressivity, capacity

Course Roadmap

- 1 **Foundations:** Approximation theory, expressivity, capacity
- 2 **Optimization:** Loss landscapes, gradient descent dynamics, implicit regularization

Course Roadmap

- 1 **Foundations:** Approximation theory, expressivity, capacity
- 2 **Optimization:** Loss landscapes, gradient descent dynamics, implicit regularization
- 3 **Generalization:** Classical bounds, modern perspectives, double descent

Course Roadmap

- 1 **Foundations:** Approximation theory, expressivity, capacity
- 2 **Optimization:** Loss landscapes, gradient descent dynamics, implicit regularization
- 3 **Generalization:** Classical bounds, modern perspectives, double descent
- 4 **Modern Frameworks:** Neural Tangent Kernel, mean field theory, information theory

Table of Contents

- 1 Introduction and Motivation
- 2 Universal Approximation Theory**
- 3 Capacity and Complexity
- 4 Optimization Theory and Loss Landscapes
- 5 Generalization Theory
- 6 Modern Theoretical Frameworks
- 7 Advanced Topics and Frontiers
- 8 Summary and Conclusion

The Fundamental Question

Can neural networks represent any function?

The Fundamental Question

Can neural networks represent any function?

The Goal

For any “well-behaved” function $f : [0, 1]^d \rightarrow \mathbb{R}$ and $\epsilon > 0$, there exists a neural network $\hat{f}$ such that:

$$\sup_{\mathbf{x} \in [0, 1]^d} |f(\mathbf{x}) - \hat{f}(\mathbf{x})| < \epsilon \quad (1)$$

Universal Approximation: Formal Statement

Theorem 1 (Universal Approximation for Shallow Networks)

Universal Approximation: Formal Statement

Theorem 1 (Universal Approximation for Shallow Networks)

Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a non-polynomial activation function (e.g., sigmoid, ReLU).

Universal Approximation: Formal Statement

Theorem 1 (Universal Approximation for Shallow Networks)

Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a non-polynomial activation function (e.g., sigmoid, ReLU).

For any continuous function $f : \mathcal{K} \rightarrow \mathbb{R}$ on a compact set $\mathcal{K} \subset \mathbb{R}^d$ and any $\epsilon > 0$, there exist:

- *Width $m \in \mathbb{N}$*
- *Weights $\mathbf{W}^{(1)} \in \mathbb{R}^{d \times m}$, $\mathbf{w}^{(2)} \in \mathbb{R}^m$*

Universal Approximation: Formal Statement

Theorem 1 (Universal Approximation for Shallow Networks)

Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a non-polynomial activation function (e.g., sigmoid, ReLU).

For any continuous function $f : \mathcal{K} \rightarrow \mathbb{R}$ on a compact set $\mathcal{K} \subset \mathbb{R}^d$ and any $\epsilon > 0$, there exist:

- *Width $m \in \mathbb{N}$*
- *Weights $\mathbf{W}^{(1)} \in \mathbb{R}^{d \times m}$, $\mathbf{w}^{(2)} \in \mathbb{R}^m$*
- *Biases $\mathbf{b}^{(1)} \in \mathbb{R}^m$, $b^{(2)} \in \mathbb{R}$*

Universal Approximation: Formal Statement

Theorem 1 (Universal Approximation for Shallow Networks)

Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a non-polynomial activation function (e.g., sigmoid, ReLU).

For any continuous function $f : \mathcal{K} \rightarrow \mathbb{R}$ on a compact set $\mathcal{K} \subset \mathbb{R}^d$ and any $\epsilon > 0$, there exist:

- Width $m \in \mathbb{N}$
- Weights $\mathbf{W}^{(1)} \in \mathbb{R}^{d \times m}$, $\mathbf{w}^{(2)} \in \mathbb{R}^m$
- Biases $\mathbf{b}^{(1)} \in \mathbb{R}^m$, $b^{(2)} \in \mathbb{R}$

such that the network

$$\hat{f}(\mathbf{x}) = b^{(2)} + \sum_{j=1}^m w_j^{(2)} \sigma \left(\sum_{i=1}^d W_{ij}^{(1)} x_i + b_j^{(1)} \right) \quad (2)$$

satisfies $\|f - \hat{f}\|_{\infty} < \epsilon$.

Universal Approximation: Intuition

“Any shape can be built from simple blocks.”

Universal Approximation: Intuition

“Any shape can be built from simple blocks.”

Proof Sketch Strategy: build localized “bump” functions.

Universal Approximation: Intuition

“Any shape can be built from simple blocks.”

Proof Sketch Strategy: build localized “bump” functions.

1 **Step 1:** Use activation σ to create a “step” or “slope”.

Universal Approximation: Intuition

“Any shape can be built from simple blocks.”

Proof Sketch Strategy: build localized “bump” functions.

- 1 **Step 1:** Use activation σ to create a “step” or “slope”.
- 2 **Step 2:** Combine two steps to create a “bump” (localized response).

Universal Approximation: Intuition

“Any shape can be built from simple blocks.”

Proof Sketch Strategy: build localized “bump” functions.

- 1 **Step 1:** Use activation σ to create a “step” or “slope”.
- 2 **Step 2:** Combine two steps to create a “bump” (localized response).
- 3 **Step 3:** Shift and scale bumps to cover the whole function.

Width vs. Depth Trade-offs

Question: The UAT says one hidden layer suffices. Why use deep networks?

Width vs. Depth Trade-offs

Question: The UAT says one hidden layer suffices. Why use deep networks?

The Curse of Width

Width vs. Depth Trade-offs

Question: The UAT says one hidden layer suffices. Why use deep networks?

The Curse of Width

- Shallow networks need *exponentially* many neurons

Width vs. Depth Trade-offs

Question: The UAT says one hidden layer suffices. Why use deep networks?

The Curse of Width

- Shallow networks need *exponentially* many neurons
- For d inputs and accuracy ϵ : $m = \mathcal{O}((1/\epsilon)^d)$

Width vs. Depth Trade-offs

Question: The UAT says one hidden layer suffices. Why use deep networks?

The Curse of Width

- Shallow networks need *exponentially* many neurons
- For d inputs and accuracy ϵ : $m = \mathcal{O}((1/\epsilon)^d)$
- Curse of dimensionality!

Width vs. Depth Trade-offs

Question: The UAT says one hidden layer suffices. Why use deep networks?

The Curse of Width

- Shallow networks need *exponentially* many neurons
- For d inputs and accuracy ϵ : $m = \mathcal{O}((1/\epsilon)^d)$
- Curse of dimensionality!

Benefits of Depth

Width vs. Depth Trade-offs

Question: The UAT says one hidden layer suffices. Why use deep networks?

The Curse of Width

- Shallow networks need *exponentially* many neurons
- For d inputs and accuracy ϵ : $m = \mathcal{O}((1/\epsilon)^d)$
- Curse of dimensionality!

Benefits of Depth

- Deep networks achieve same approximation with *polynomial* parameters

Width vs. Depth Trade-offs

Question: The UAT says one hidden layer suffices. Why use deep networks?

The Curse of Width

- Shallow networks need *exponentially* many neurons
- For d inputs and accuracy ϵ : $m = \mathcal{O}((1/\epsilon)^d)$
- Curse of dimensionality!

Benefits of Depth

- Deep networks achieve same approximation with *polynomial* parameters
- Hierarchical feature learning

Width vs. Depth Trade-offs

Question: The UAT says one hidden layer suffices. Why use deep networks?

The Curse of Width

- Shallow networks need *exponentially* many neurons
- For d inputs and accuracy ϵ : $m = \mathcal{O}((1/\epsilon)^d)$
- Curse of dimensionality!

Benefits of Depth

- Deep networks achieve same approximation with *polynomial* parameters
- Hierarchical feature learning
- Compositional structure matches many real-world functions

Expressivity: Depth Separation Results

Theorem 2 (Depth Separation (Simplified))

There exist functions $f : \mathbb{R}^d \rightarrow \mathbb{R}$ such that:

Expressivity: Depth Separation Results

Theorem 2 (Depth Separation (Simplified))

There exist functions $f : \mathbb{R}^d \rightarrow \mathbb{R}$ such that:

- *A deep network with $\mathcal{O}(d)$ layers can represent f exactly.*

Expressivity: Depth Separation Results

Theorem 2 (Depth Separation (Simplified))

There exist functions $f : \mathbb{R}^d \rightarrow \mathbb{R}$ such that:

- *A deep network with $\mathcal{O}(d)$ layers can represent f exactly.*
- *A shallow network (one hidden layer) needs $\exp(d)$ neurons to achieve the same accuracy.*

Expressivity: Depth Separation Results

Theorem 2 (Depth Separation (Simplified))

There exist functions $f : \mathbb{R}^d \rightarrow \mathbb{R}$ such that:

- *A deep network with $\mathcal{O}(d)$ layers can represent f exactly.*
- *A shallow network (one hidden layer) needs $\exp(d)$ neurons to achieve the same accuracy.*

Examples of such functions:

Expressivity: Depth Separation Results

Theorem 2 (Depth Separation (Simplified))

There exist functions $f : \mathbb{R}^d \rightarrow \mathbb{R}$ such that:

- *A deep network with $\mathcal{O}(d)$ layers can represent f exactly.*
- *A shallow network (one hidden layer) needs $\exp(d)$ neurons to achieve the same accuracy.*

Examples of such functions:

- Boolean circuits (parity, majority)

Expressivity: Depth Separation Results

Theorem 2 (Depth Separation (Simplified))

There exist functions $f : \mathbb{R}^d \rightarrow \mathbb{R}$ such that:

- *A deep network with $\mathcal{O}(d)$ layers can represent f exactly.*
- *A shallow network (one hidden layer) needs $\exp(d)$ neurons to achieve the same accuracy.*

Examples of such functions:

- Boolean circuits (parity, majority)
- Compositional functions: $f = f_L \circ f_{L-1} \circ \cdots \circ f_1$

Expressivity: Depth Separation Results

Theorem 2 (Depth Separation (Simplified))

There exist functions $f : \mathbb{R}^d \rightarrow \mathbb{R}$ such that:

- *A deep network with $\mathcal{O}(d)$ layers can represent f exactly.*
- *A shallow network (one hidden layer) needs $\exp(d)$ neurons to achieve the same accuracy.*

Examples of such functions:

- Boolean circuits (parity, majority)
- Compositional functions: $f = f_L \circ f_{L-1} \circ \cdots \circ f_1$
- Modular arithmetic

Table of Contents

- 1 Introduction and Motivation
- 2 Universal Approximation Theory
- 3 Capacity and Complexity**
- 4 Optimization Theory and Loss Landscapes
- 5 Generalization Theory
- 6 Modern Theoretical Frameworks
- 7 Advanced Topics and Frontiers
- 8 Summary and Conclusion

From Expressivity to Capacity

Universal approximation tells us networks *can* represent functions.

From Expressivity to Capacity

Universal approximation tells us networks *can* represent functions.

But for learning, we need to understand:

From Expressivity to Capacity

Universal approximation tells us networks *can* represent functions.

But for learning, we need to understand:

- How many functions can a network of given size represent?

From Expressivity to Capacity

Universal approximation tells us networks *can* represent functions.

But for learning, we need to understand:

- How many functions can a network of given size represent?
- What is the effective “capacity” of the hypothesis class?

From Expressivity to Capacity

Universal approximation tells us networks *can* represent functions.

But for learning, we need to understand:

- How many functions can a network of given size represent?
- What is the effective “capacity” of the hypothesis class?
- How does capacity relate to generalization?

From Expressivity to Capacity

Universal approximation tells us networks *can* represent functions.

But for learning, we need to understand:

- How many functions can a network of given size represent?
- What is the effective “capacity” of the hypothesis class?
- How does capacity relate to generalization?

Classical Learning Theory Tools

- VC Dimension
- Rademacher Complexity
- Fat-shattering Dimension

VC Dimension

Definition 3 (VC Dimension)

The VC dimension of a hypothesis class $\mathcal{H}$ is the maximum number of points that can be *shattered* by $\mathcal{H}$.

VC Dimension

Definition 3 (VC Dimension)

The VC dimension of a hypothesis class $\mathcal{H}$ is the maximum number of points that can be *shattered* by $\mathcal{H}$.

Shattering

A set of points $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ is **shattered** if for every labeling $\{y_1, \dots, y_n\} \in \{0, 1\}^n$, there exists $h \in \mathcal{H}$ such that $h(\mathbf{x}_i) = y_i$ for all i .

VC Dimension

Definition 3 (VC Dimension)

The VC dimension of a hypothesis class $\mathcal{H}$ is the maximum number of points that can be *shattered* by $\mathcal{H}$.

Shattering

A set of points $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ is **shattered** if for every labeling $\{y_1, \dots, y_n\} \in \{0, 1\}^n$, there exists $h \in \mathcal{H}$ such that $h(\mathbf{x}_i) = y_i$ for all i .

Example: Linear Classifiers in $\mathbb{R}^2$

VC dimension = 3. Can shatter any 3 non-collinear points, but not all sets of 4.

VC Dimension of Neural Networks

Theorem 4 (VC Dimension Upper Bound)

For a neural network with W weights (parameters), the VC dimension satisfies:

$$VC\text{-dim} = \mathcal{O}(W \log W) \quad (3)$$

VC Dimension of Neural Networks

Theorem 4 (VC Dimension Upper Bound)

For a neural network with W weights (parameters), the VC dimension satisfies:

$$VC\text{-dim} = \mathcal{O}(W \log W) \quad (3)$$

Implications:

VC Dimension of Neural Networks

Theorem 4 (VC Dimension Upper Bound)

For a neural network with W weights (parameters), the VC dimension satisfies:

$$VC\text{-dim} = \mathcal{O}(W \log W) \quad (3)$$

Implications:

- VC dimension grows roughly *linearly* with number of parameters

VC Dimension of Neural Networks

Theorem 4 (VC Dimension Upper Bound)

For a neural network with W weights (parameters), the VC dimension satisfies:

$$VC\text{-dim} = \mathcal{O}(W \log W) \quad (3)$$

Implications:

- VC dimension grows roughly *linearly* with number of parameters
- Suggests sample complexity $\tilde{\mathcal{O}}(W)$ for PAC learning

VC Dimension of Neural Networks

Theorem 4 (VC Dimension Upper Bound)

For a neural network with W weights (parameters), the VC dimension satisfies:

$$VC\text{-dim} = \mathcal{O}(W \log W) \quad (3)$$

Implications:

- VC dimension grows roughly *linearly* with number of parameters
- Suggests sample complexity $\tilde{\mathcal{O}}(W)$ for PAC learning
- But modern overparameterized networks have $W \gg n$!

VC Dimension of Neural Networks

Theorem 4 (VC Dimension Upper Bound)

For a neural network with W weights (parameters), the VC dimension satisfies:

$$VC\text{-dim} = \mathcal{O}(W \log W) \quad (3)$$

Implications:

- VC dimension grows roughly *linearly* with number of parameters
- Suggests sample complexity $\tilde{\mathcal{O}}(W)$ for PAC learning
- But modern overparameterized networks have $W \gg n$!

The Mystery

Classical VC theory predicts poor generalization for overparameterized networks, but they work remarkably well in practice!

Rademacher Complexity

Definition 5 (Empirical Rademacher Complexity)

For a hypothesis class $\mathcal{H}$ and dataset $\mathcal{S} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$:

$$\hat{\mathfrak{R}}_{\mathcal{S}}(\mathcal{H}) = \mathbb{E} \left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \sigma_i h(\mathbf{x}_i) \right] \quad (4)$$

where $\sigma_i \in \{+1, -1\}$ are independent random signs (Rademacher variables).

Rademacher Complexity

Definition 5 (Empirical Rademacher Complexity)

For a hypothesis class $\mathcal{H}$ and dataset $\mathcal{S} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$:

$$\hat{\mathfrak{R}}_{\mathcal{S}}(\mathcal{H}) = \mathbb{E} \left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \sigma_i h(\mathbf{x}_i) \right] \quad (4)$$

where $\sigma_i \in \{+1, -1\}$ are independent random signs (Rademacher variables).

Intuition:

Rademacher Complexity

Definition 5 (Empirical Rademacher Complexity)

For a hypothesis class $\mathcal{H}$ and dataset $\mathcal{S} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$:

$$\hat{\mathfrak{R}}_{\mathcal{S}}(\mathcal{H}) = \mathbb{E} \left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \sigma_i h(\mathbf{x}_i) \right] \quad (4)$$

where $\sigma_i \in \{+1, -1\}$ are independent random signs (Rademacher variables).

Intuition:

- Measures how well $\mathcal{H}$ can fit random noise

Rademacher Complexity

Definition 5 (Empirical Rademacher Complexity)

For a hypothesis class $\mathcal{H}$ and dataset $\mathcal{S} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$:

$$\hat{\mathfrak{R}}_{\mathcal{S}}(\mathcal{H}) = \mathbb{E} \left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \sigma_i h(\mathbf{x}_i) \right] \quad (4)$$

where $\sigma_i \in \{+1, -1\}$ are independent random signs (Rademacher variables).

Intuition:

- Measures how well $\mathcal{H}$ can fit random noise
- Higher complexity = can fit more random patterns

Rademacher Complexity

Definition 5 (Empirical Rademacher Complexity)

For a hypothesis class $\mathcal{H}$ and dataset $\mathcal{S} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$:

$$\hat{\mathfrak{R}}_{\mathcal{S}}(\mathcal{H}) = \mathbb{E} \left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \sigma_i h(\mathbf{x}_i) \right] \quad (4)$$

where $\sigma_i \in \{+1, -1\}$ are independent random signs (Rademacher variables).

Intuition:

- Measures how well $\mathcal{H}$ can fit random noise
- Higher complexity = can fit more random patterns
- Relates directly to generalization gap

Rademacher Complexity: Generalization Bound

Theorem 6 (Rademacher Generalization Bound)

With probability at least $1 - \delta$:

$$\mathbb{E} [\ell(h)] \leq \frac{1}{n} \sum_{i=1}^n \ell(h, \mathbf{x}_i) + 2\mathfrak{R}_n(\mathcal{H}) + \sqrt{\frac{\log(1/\delta)}{2n}} \quad (5)$$

Rademacher Complexity: Generalization Bound

Theorem 6 (Rademacher Generalization Bound)

With probability at least $1 - \delta$:

$$\mathbb{E} [\ell(h)] \leq \frac{1}{n} \sum_{i=1}^n \ell(h, \mathbf{x}_i) + 2\mathfrak{R}_n(\mathcal{H}) + \sqrt{\frac{\log(1/\delta)}{2n}} \quad (5)$$

For neural networks:

Rademacher Complexity: Generalization Bound

Theorem 6 (Rademacher Generalization Bound)

With probability at least $1 - \delta$:

$$\mathbb{E} [\ell(h)] \leq \frac{1}{n} \sum_{i=1}^n \ell(h, \mathbf{x}_i) + 2\mathfrak{R}_n(\mathcal{H}) + \sqrt{\frac{\log(1/\delta)}{2n}} \quad (5)$$

For neural networks:

- Rademacher complexity depends on norm of weight matrices

Rademacher Complexity: Generalization Bound

Theorem 6 (Rademacher Generalization Bound)

With probability at least $1 - \delta$:

$$\mathbb{E} [\ell(h)] \leq \frac{1}{n} \sum_{i=1}^n \ell(h, \mathbf{x}_i) + 2\mathfrak{R}_n(\mathcal{H}) + \sqrt{\frac{\log(1/\delta)}{2n}} \quad (5)$$

For neural networks:

- Rademacher complexity depends on norm of weight matrices
- Smaller norms $\Rightarrow$ better generalization

Rademacher Complexity: Generalization Bound

Theorem 6 (Rademacher Generalization Bound)

With probability at least $1 - \delta$:

$$\mathbb{E} [\ell(h)] \leq \frac{1}{n} \sum_{i=1}^n \ell(h, \mathbf{x}_i) + 2\mathfrak{R}_n(\mathcal{H}) + \sqrt{\frac{\log(1/\delta)}{2n}} \quad (5)$$

For neural networks:

- Rademacher complexity depends on norm of weight matrices
- Smaller norms $\Rightarrow$ better generalization
- Suggests implicit regularization during training

Table of Contents

- 1 Introduction and Motivation
- 2 Universal Approximation Theory
- 3 Capacity and Complexity
- 4 Optimization Theory and Loss Landscapes**
 - Gradient Descent Dynamics
 - Implicit Regularization
- 5 Generalization Theory
- 6 Modern Theoretical Frameworks
- 7 Advanced Topics and Frontiers

The Optimization Challenge

Training objective:

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(\mathbf{x}_i), y_i) \quad (6)$$

The Optimization Challenge

Training objective:

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(\mathbf{x}_i), y_i) \quad (6)$$

Fundamental Challenges

The Optimization Challenge

Training objective:

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(\mathbf{x}_i), y_i) \quad (6)$$

Fundamental Challenges

- **Non-convex:** Many local minima, saddle points

The Optimization Challenge

Training objective:

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(\mathbf{x}_i), y_i) \quad (6)$$

Fundamental Challenges

- **Non-convex:** Many local minima, saddle points
- **High-dimensional:** Millions/billions of parameters

The Optimization Challenge

Training objective:

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(\mathbf{x}_i), y_i) \quad (6)$$

Fundamental Challenges

- **Non-convex:** Many local minima, saddle points
- **High-dimensional:** Millions/billions of parameters
- **No global guarantees:** Yet SGD works remarkably well!

The Optimization Challenge

Training objective:

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(\mathbf{x}_i), y_i) \quad (6)$$

Fundamental Challenges

- **Non-convex:** Many local minima, saddle points
- **High-dimensional:** Millions/billions of parameters
- **No global guarantees:** Yet SGD works remarkably well!

Key Questions:

The Optimization Challenge

Training objective:

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(\mathbf{x}_i), y_i) \quad (6)$$

Fundamental Challenges

- **Non-convex:** Many local minima, saddle points
- **High-dimensional:** Millions/billions of parameters
- **No global guarantees:** Yet SGD works remarkably well!

Key Questions:

- What does the loss landscape look like?

The Optimization Challenge

Training objective:

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(\mathbf{x}_i), y_i) \quad (6)$$

Fundamental Challenges

- **Non-convex:** Many local minima, saddle points
- **High-dimensional:** Millions/billions of parameters
- **No global guarantees:** Yet SGD works remarkably well!

Key Questions:

- What does the loss landscape look like?
- Why does gradient descent find good solutions?

The Optimization Challenge

Training objective:

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(\mathbf{x}_i), y_i) \quad (6)$$

Fundamental Challenges

- **Non-convex:** Many local minima, saddle points
- **High-dimensional:** Millions/billions of parameters
- **No global guarantees:** Yet SGD works remarkably well!

Key Questions:

- What does the loss landscape look like?
- Why does gradient descent find good solutions?
- What role does overparameterization play?

Loss Landscape Visualization

Loss Landscape Visualization

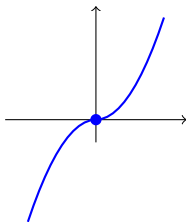
Key features:

- Multiple local minima
- Many saddle points (exponentially many in high dimensions)
- Flat regions and sharp valleys

The Geometry of High-Dimensional Landscapes

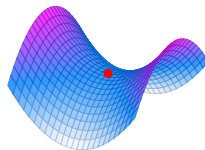
In high dimensions, the “energy” is concentrated in saddle points.

Local Minimum



All d eigenvalues of Hessian > 0 . Probability $\approx (1/2)^d$.

Saddle Point

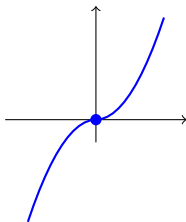


Some eigenvalues > 0 , some < 0 .
Exponentially more common!

The Geometry of High-Dimensional Landscapes

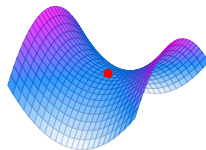
In high dimensions, the “energy” is concentrated in saddle points.

Local Minimum



All d eigenvalues of Hessian > 0 . Probability $\approx (1/2)^d$.

Saddle Point



Some eigenvalues > 0 , some < 0 .
Exponentially more common!

Random non-convex landscapes in high dimensions are mostly flat with many saddle points. “Bad” local minima are rare.

Why SGD Escapes Saddle Points

- **Curvature:** Saddle points have “downward” directions (negative eigenvalues).

Why SGD Escapes Saddle Points

- **Curvature:** Saddle points have “downward” directions (negative eigenvalues).
- **Noise helps (STOCHASTIC):** The gradient noise in SGD helps “push” the parameters into these downward directions.

Why SGD Escapes Saddle Points

- **Curvature:** Saddle points have “downward” directions (negative eigenvalues).
- **Noise helps (STOCHASTIC):** The gradient noise in SGD helps “push” the parameters into these downward directions.
- **Escaping time:** GD with random initialization or SGD escapes saddle points in polynomial time.

Overparameterization and the Loss Landscape

Theorem 7 (Informal: Overparameterization Helps)

For sufficiently overparameterized networks (width $m \gg n$):

Overparameterization and the Loss Landscape

Theorem 7 (Informal: Overparameterization Helps)

For sufficiently overparameterized networks (width $m \gg n$):

- *The loss landscape has no “bad” local minima*

Overparameterization and the Loss Landscape

Theorem 7 (Informal: Overparameterization Helps)

For sufficiently overparameterized networks (width $m \gg n$):

- *The loss landscape has no “bad” local minima*
- *All local minima achieve near-zero training loss*

Overparameterization and the Loss Landscape

Theorem 7 (Informal: Overparameterization Helps)

For sufficiently overparameterized networks (width $m \gg n$):

- *The loss landscape has no “bad” local minima*
- *All local minima achieve near-zero training loss*
- *Many global minima exist*

Overparameterization and the Loss Landscape

Theorem 7 (Informal: Overparameterization Helps)

For sufficiently overparameterized networks (width $m \gg n$):

- *The loss landscape has no “bad” local minima*
- *All local minima achieve near-zero training loss*
- *Many global minima exist*

Intuition:

Overparameterization and the Loss Landscape

Theorem 7 (Informal: Overparameterization Helps)

For sufficiently overparameterized networks (width $m \gg n$):

- *The loss landscape has no “bad” local minima*
- *All local minima achieve near-zero training loss*
- *Many global minima exist*

Intuition:

- More parameters = more degrees of freedom

Overparameterization and the Loss Landscape

Theorem 7 (Informal: Overparameterization Helps)

For sufficiently overparameterized networks (width $m \gg n$):

- *The loss landscape has no “bad” local minima*
- *All local minima achieve near-zero training loss*
- *Many global minima exist*

Intuition:

- More parameters = more degrees of freedom
- Easier to find interpolating solutions ($\mathcal{L}_{\text{train}} \approx 0$)

Overparameterization and the Loss Landscape

Theorem 7 (Informal: Overparameterization Helps)

For sufficiently overparameterized networks (width $m \gg n$):

- *The loss landscape has no “bad” local minima*
- *All local minima achieve near-zero training loss*
- *Many global minima exist*

Intuition:

- More parameters = more degrees of freedom
- Easier to find interpolating solutions ($\mathcal{L}_{\text{train}} \approx 0$)
- Loss landscape becomes “friendlier”

Table of Contents

- 1 Introduction and Motivation
- 2 Universal Approximation Theory
- 3 Capacity and Complexity
- 4 Optimization Theory and Loss Landscapes**
 - **Gradient Descent Dynamics**
 - Implicit Regularization
- 5 Generalization Theory
- 6 Modern Theoretical Frameworks
 - Neural Tangent Kernel (NTK)
 - Mean-Field and Information Theory
- 7 Advanced Topics and Frontiers
 - Robustness and Adversarial Examples
 - Theory of Generative Models
- 8 Summary and Conclusion

Convergence Analysis: Smooth Functions

Assumption 1 (L -Smoothness)

The loss function $\mathcal{L}$ is L -smooth if:

$$\|\nabla \mathcal{L}(\theta_1) - \nabla \mathcal{L}(\theta_2)\| \leq L \|\theta_1 - \theta_2\| \quad (7)$$

Convergence Analysis: Smooth Functions

Assumption 1 (L -Smoothness)

The loss function $\mathcal{L}$ is L -smooth if:

$$\|\nabla \mathcal{L}(\theta_1) - \nabla \mathcal{L}(\theta_2)\| \leq L \|\theta_1 - \theta_2\| \quad (7)$$

Theorem 8 (GD Convergence for Smooth Non-convex)

For gradient descent with step size $\eta \leq 1/L$:

$$\min_{t=0,\dots,T-1} \|\nabla \mathcal{L}(\theta_t)\|^2 \leq \frac{2L(\mathcal{L}(\theta_0) - \mathcal{L}^*)}{T} \quad (8)$$

Convergence Analysis: Smooth Functions

Assumption 1 (L -Smoothness)

The loss function $\mathcal{L}$ is L -smooth if:

$$\|\nabla \mathcal{L}(\theta_1) - \nabla \mathcal{L}(\theta_2)\| \leq L \|\theta_1 - \theta_2\| \quad (7)$$

Theorem 8 (GD Convergence for Smooth Non-convex)

For gradient descent with step size $\eta \leq 1/L$:

$$\min_{t=0,\dots,T-1} \|\nabla \mathcal{L}(\theta_t)\|^2 \leq \frac{2L(\mathcal{L}(\theta_0) - \mathcal{L}^*)}{T} \quad (8)$$

Interpretation:

Convergence Analysis: Smooth Functions

Assumption 1 (L -Smoothness)

The loss function $\mathcal{L}$ is L -smooth if:

$$\|\nabla \mathcal{L}(\theta_1) - \nabla \mathcal{L}(\theta_2)\| \leq L \|\theta_1 - \theta_2\| \quad (7)$$

Theorem 8 (GD Convergence for Smooth Non-convex)

For gradient descent with step size $\eta \leq 1/L$:

$$\min_{t=0,\dots,T-1} \|\nabla \mathcal{L}(\theta_t)\|^2 \leq \frac{2L(\mathcal{L}(\theta_0) - \mathcal{L}^*)}{T} \quad (8)$$

Interpretation:

- GD finds stationary point ($\nabla \mathcal{L} \approx 0$) in $\mathcal{O}(1/\epsilon^2)$ iterations

Convergence Analysis: Smooth Functions

Assumption 1 (L -Smoothness)

The loss function $\mathcal{L}$ is L -smooth if:

$$\|\nabla \mathcal{L}(\theta_1) - \nabla \mathcal{L}(\theta_2)\| \leq L \|\theta_1 - \theta_2\| \quad (7)$$

Theorem 8 (GD Convergence for Smooth Non-convex)

For gradient descent with step size $\eta \leq 1/L$:

$$\min_{t=0,\dots,T-1} \|\nabla \mathcal{L}(\theta_t)\|^2 \leq \frac{2L(\mathcal{L}(\theta_0) - \mathcal{L}^*)}{T} \quad (8)$$

Interpretation:

- GD finds stationary point ($\nabla \mathcal{L} \approx 0$) in $\mathcal{O}(1/\epsilon^2)$ iterations
- No guarantee it's a global minimum!

Convergence Analysis: Smooth Functions

Assumption 1 (L -Smoothness)

The loss function $\mathcal{L}$ is L -smooth if:

$$\|\nabla \mathcal{L}(\theta_1) - \nabla \mathcal{L}(\theta_2)\| \leq L \|\theta_1 - \theta_2\| \quad (7)$$

Theorem 8 (GD Convergence for Smooth Non-convex)

For gradient descent with step size $\eta \leq 1/L$:

$$\min_{t=0, \dots, T-1} \|\nabla \mathcal{L}(\theta_t)\|^2 \leq \frac{2L(\mathcal{L}(\theta_0) - \mathcal{L}^*)}{T} \quad (8)$$

Interpretation:

- GD finds stationary point ($\nabla \mathcal{L} \approx 0$) in $\mathcal{O}(1/\epsilon^2)$ iterations
- No guarantee it's a global minimum!
- In high dimensions, most “bad” local minima disappear (overparameterization).

Gradient Flow Perspective

Consider continuous-time limit of gradient descent (as $\eta \rightarrow 0$):

$$\frac{d\theta(t)}{dt} = -\nabla \mathcal{L}(\theta(t)) \quad (9)$$

Gradient Flow Perspective

Consider continuous-time limit of gradient descent (as $\eta \rightarrow 0$):

$$\frac{d\theta(t)}{dt} = -\nabla \mathcal{L}(\theta(t)) \quad (9)$$

Why Gradient Flow?

Gradient Flow Perspective

Consider continuous-time limit of gradient descent (as $\eta \rightarrow 0$):

$$\frac{d\theta(t)}{dt} = -\nabla \mathcal{L}(\theta(t)) \quad (9)$$

Why Gradient Flow?

- Tool for rigorous mathematical proofs

Gradient Flow Perspective

Consider continuous-time limit of gradient descent (as $\eta \rightarrow 0$):

$$\frac{d\theta(t)}{dt} = -\nabla \mathcal{L}(\theta(t)) \quad (9)$$

Why Gradient Flow?

- Tool for rigorous mathematical proofs
- Connection to dynamical systems (stability, convergence)

Gradient Flow Perspective

Consider continuous-time limit of gradient descent (as $\eta \rightarrow 0$):

$$\frac{d\theta(t)}{dt} = -\nabla \mathcal{L}(\theta(t)) \quad (9)$$

Why Gradient Flow?

- Tool for rigorous mathematical proofs
- Connection to dynamical systems (stability, convergence)
- Reveals **implicit biases** of the optimization path

Gradient Flow Perspective

Consider continuous-time limit of gradient descent (as $\eta \rightarrow 0$):

$$\frac{d\theta(t)}{dt} = -\nabla \mathcal{L}(\theta(t)) \quad (9)$$

Why Gradient Flow?

- Tool for rigorous mathematical proofs
- Connection to dynamical systems (stability, convergence)
- Reveals **implicit biases** of the optimization path

Key Insight

The *path* taken by gradient descent is as important as the final point, as it selects solutions with specific structural properties.

Table of Contents

- 1 Introduction and Motivation
- 2 Universal Approximation Theory
- 3 Capacity and Complexity
- 4 Optimization Theory and Loss Landscapes**
 - Gradient Descent Dynamics
 - **Implicit Regularization**
- 5 Generalization Theory
- 6 Modern Theoretical Frameworks
 - Neural Tangent Kernel (NTK)
 - Mean-Field and Information Theory
- 7 Advanced Topics and Frontiers
 - Robustness and Adversarial Examples
 - Theory of Generative Models
- 8 Summary and Conclusion

The Implicit Regularization Phenomenon

Why does gradient descent generalize well?

The Implicit Regularization Phenomenon

Why does gradient descent generalize well?

Observation: Overparameterized networks have many global minima (zero training error), but they generalize differently.

The Implicit Regularization Phenomenon

Why does gradient descent generalize well?

Observation: Overparameterized networks have many global minima (zero training error), but they generalize differently.

Implicit Regularization

The optimization algorithm (SGD/GD) *implicitly* prefers “simple” or “flat” solutions, effectively regularizing the model without explicit penalties.

Case Study: Deep Linear Networks

Consider a network $f(\mathbf{x}) = \mathbf{W}_L \mathbf{W}_{L-1} \cdots \mathbf{W}_1 \mathbf{x}$.

Case Study: Deep Linear Networks

Consider a network $f(\mathbf{x}) = \mathbf{W}_L \mathbf{W}_{L-1} \cdots \mathbf{W}_1 \mathbf{x}$.

- **Expressivity:** It's just a linear map $\mathbf{x} \mapsto \mathbf{W}_{eff} \mathbf{x}$.

Case Study: Deep Linear Networks

Consider a network $f(\mathbf{x}) = \mathbf{W}_L \mathbf{W}_{L-1} \cdots \mathbf{W}_1 \mathbf{x}$.

- **Expressivity:** It's just a linear map $\mathbf{x} \mapsto \mathbf{W}_{eff} \mathbf{x}$.
- **Optimization:** The loss $\mathcal{L}(\mathbf{W}_1, \dots, \mathbf{W}_L)$ is **non-convex**.

Case Study: Deep Linear Networks

Consider a network $f(\mathbf{x}) = \mathbf{W}_L \mathbf{W}_{L-1} \cdots \mathbf{W}_1 \mathbf{x}$.

- **Expressivity:** It's just a linear map $\mathbf{x} \mapsto \mathbf{W}_{eff} \mathbf{x}$.
- **Optimization:** The loss $\mathcal{L}(\mathbf{W}_1, \dots, \mathbf{W}_L)$ is **non-convex**.
- **The Mystery:** Why use multiple layers if the end result is linear?

Case Study: Deep Linear Networks

Consider a network $f(\mathbf{x}) = \mathbf{W}_L \mathbf{W}_{L-1} \cdots \mathbf{W}_1 \mathbf{x}$.

- **Expressivity:** It's just a linear map $\mathbf{x} \mapsto \mathbf{W}_{eff} \mathbf{x}$.
- **Optimization:** The loss $\mathcal{L}(\mathbf{W}_1, \dots, \mathbf{W}_L)$ is **non-convex**.
- **The Mystery:** Why use multiple layers if the end result is linear?

The Power of Depth (in Optimization)

Depth changes the **optimization dynamics**. Even though the final function is linear, the trajectory taken by GD favors **low-rank** solutions.

Implicit Bias: Matrix Factoring Perspective

$$\min_{\mathbf{W}_1, \mathbf{W}_2} \|\mathbf{Y} - \mathbf{W}_2 \mathbf{W}_1 \mathbf{X}\|_F^2$$

Gradient Flow Results:

Implicit Bias: Matrix Factoring Perspective

$$\min_{\mathbf{W}_1, \mathbf{W}_2} \|\mathbf{Y} - \mathbf{W}_2 \mathbf{W}_1 \mathbf{X}\|_F^2$$

Gradient Flow Results:

- With **small initialization**, GD stays near the manifold of low-rank matrices.

Implicit Bias: Matrix Factoring Perspective

$$\min_{\mathbf{W}_1, \mathbf{W}_2} \|\mathbf{Y} - \mathbf{W}_2 \mathbf{W}_1 \mathbf{X}\|_F^2$$

Gradient Flow Results:

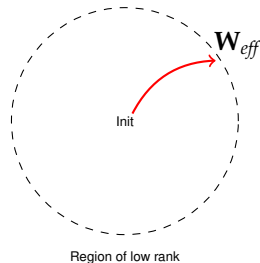
- With **small initialization**, GD stays near the manifold of low-rank matrices.
- It acts like *nuclear norm regularization* without an explicit penalty.

Implicit Bias: Matrix Factoring Perspective

$$\min_{\mathbf{W}_1, \mathbf{W}_2} \|\mathbf{Y} - \mathbf{W}_2 \mathbf{W}_1 \mathbf{X}\|_F^2$$

Gradient Flow Results:

- With **small initialization**, GD stays near the manifold of low-rank matrices.
- It acts like *nuclear norm regularization* without an explicit penalty.

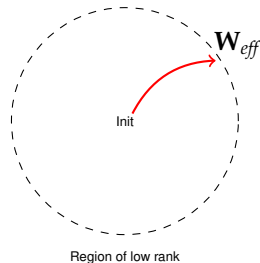


Implicit Bias: Matrix Factoring Perspective

$$\min_{\mathbf{W}_1, \mathbf{W}_2} \|\mathbf{Y} - \mathbf{W}_2 \mathbf{W}_1 \mathbf{X}\|_F^2$$

Gradient Flow Results:

- With **small initialization**, GD stays near the manifold of low-rank matrices.
- It acts like *nuclear norm regularization* without an explicit penalty.



The overparameterization in deep linear networks provides a “steering mechanism” for optimization.

Edge of Stability (EoS)

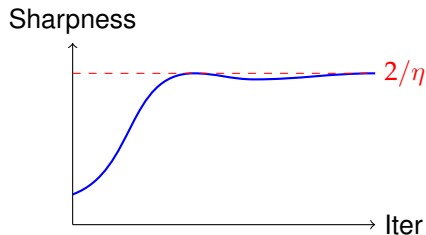
- Classical optimization theory suggests $\eta < 2/L$ for stability.

Edge of Stability (EoS)

- Classical optimization theory suggests $\eta < 2/L$ for stability.
- **Discovery (Cohen et al. 2021):**
Deep learning training often happens where **sharpness** $\lambda_{\max}(\nabla^2 \mathcal{L}) \approx 2/\eta$.

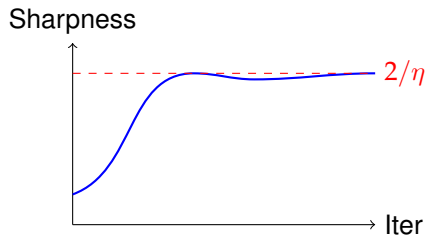
Edge of Stability (EoS)

- Classical optimization theory suggests $\eta < 2/L$ for stability.
- **Discovery (Cohen et al. 2021):**
Deep learning training often happens where **sharpness** $\lambda_{\max}(\nabla^2 \mathcal{L}) \approx 2/\eta$.



Edge of Stability (EoS)

- Classical optimization theory suggests $\eta < 2/L$ for stability.
- **Discovery (Cohen et al. 2021):**
Deep learning training often happens where **sharpness** $\lambda_{\max}(\nabla^2 \mathcal{L}) \approx 2/\eta$.



Implication

Neural networks are trained at the “edge” of divergence, which may provide further implicit regularization by pushing the model out of sharp minima.

Table of Contents

- 1 Introduction and Motivation
- 2 Universal Approximation Theory
- 3 Capacity and Complexity
- 4 Optimization Theory and Loss Landscapes
- 5 Generalization Theory**
- 6 Modern Theoretical Frameworks
- 7 Advanced Topics and Frontiers
- 8 Summary and Conclusion

The Generalization Puzzle

Why do overparameterized networks generalize?

The Generalization Puzzle

Why do overparameterized networks generalize?

Classical Wisdom (Bias-Variance Trade-off):

The Generalization Puzzle

Why do overparameterized networks generalize?

Classical Wisdom (Bias-Variance Trade-off):

- **Underfitting:** Model too simple, high bias.

The Generalization Puzzle

Why do overparameterized networks generalize?

Classical Wisdom (Bias-Variance Trade-off):

- **Underfitting:** Model too simple, high bias.
- **Overfitting:** Model too complex, high variance.

The Generalization Puzzle

Why do overparameterized networks generalize?

Classical Wisdom (Bias-Variance Trade-off):

- **Underfitting:** Model too simple, high bias.
- **Overfitting:** Model too complex, high variance.
- **Sweet Spot:** Matches data complexity (Occam's Razor).

The Generalization Puzzle

Why do overparameterized networks generalize?

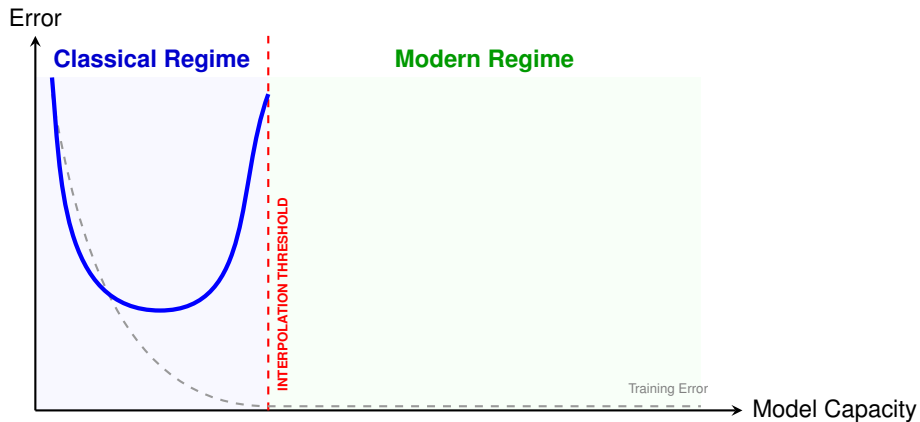
Classical Wisdom (Bias-Variance Trade-off):

- **Underfitting:** Model too simple, high bias.
- **Overfitting:** Model too complex, high variance.
- **Sweet Spot:** Matches data complexity (Occam's Razor).

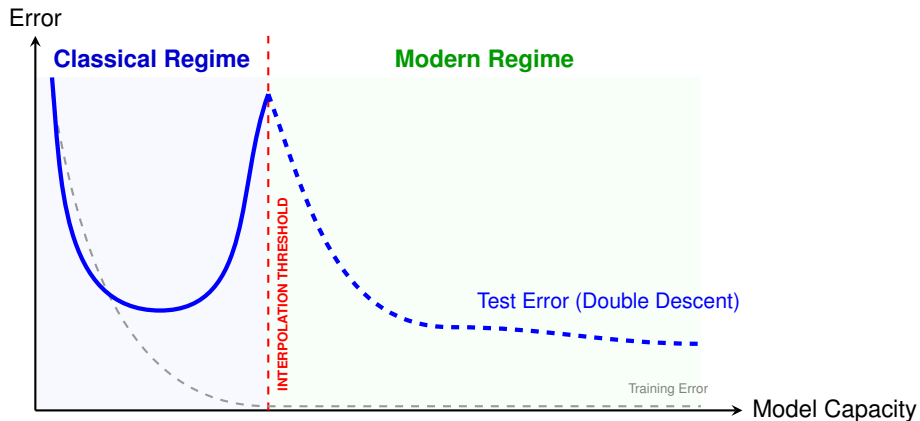
The Modern Mystery

Deep learning models often have $10\times$ more parameters than data points.
They can fit random labels, yet they generalize better as they get larger!

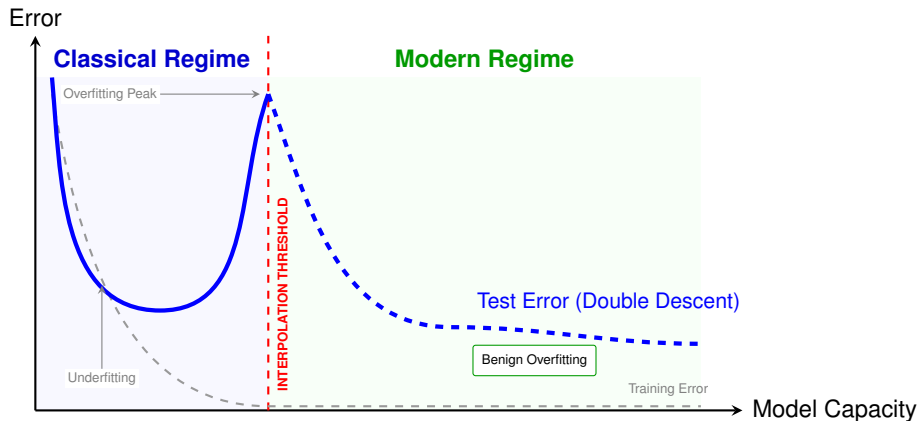
The Double Descent Phenomenon



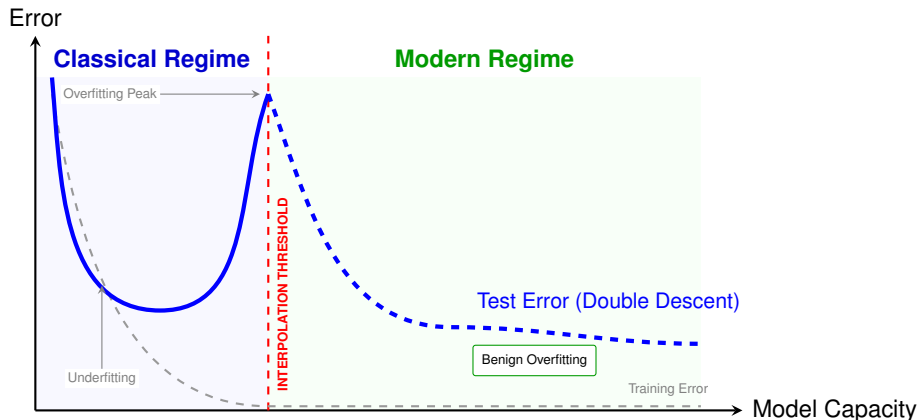
The Double Descent Phenomenon



The Double Descent Phenomenon



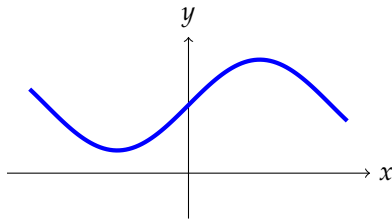
The Double Descent Phenomenon



Beyond the interpolation threshold, increasing capacity actually **improves** generalization.
Larger models are “more regularized” by SGD.

Benign Overfitting: High-Dimensional “Noise Hiding”

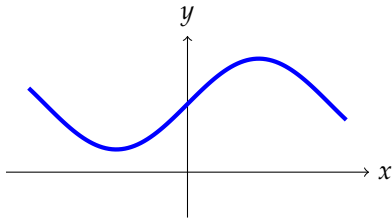
How can we fit noise perfectly without ruining the signal?



Benign Overfitting: High-Dimensional “Noise Hiding”

How can we fit noise perfectly without ruining the signal?

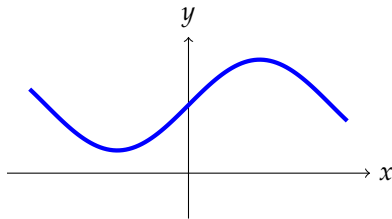
- **The Phenomenon:** Perfect interpolation of noisy data leads to good prediction on test data.



Benign Overfitting: High-Dimensional “Noise Hiding”

How can we fit noise perfectly without ruining the signal?

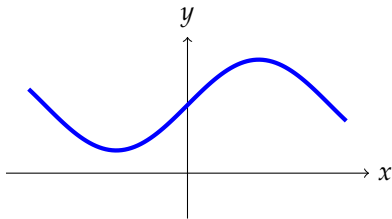
- **The Phenomenon:** Perfect interpolation of noisy data leads to good prediction on test data.
- **High-D Mechanism:** In high dimensions ($d \gg n$), there are many “extra” directions.



Benign Overfitting: High-Dimensional “Noise Hiding”

How can we fit noise perfectly without ruining the signal?

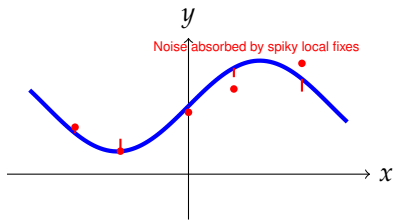
- **The Phenomenon:** Perfect interpolation of noisy data leads to good prediction on test data.
- **High-D Mechanism:** In high dimensions ($d \gg n$), there are many “extra” directions.
- The model “absorbs” label noise into high-frequency, low-energy components of the function.



Benign Overfitting: High-Dimensional “Noise Hiding”

How can we fit noise perfectly without ruining the signal?

- **The Phenomenon:** Perfect interpolation of noisy data leads to good prediction on test data.
- **High-D Mechanism:** In high dimensions ($d \gg n$), there are many “extra” directions.
- The model “absorbs” label noise into high-frequency, low-energy components of the function.



Inductive Biases: The “Real” Capacity

Effective Capacity $\neq$ Number of Parameters.

What limits complexity?

Inductive Biases: The “Real” Capacity

Effective Capacity $\neq$ Number of Parameters.

What limits complexity?

- **Norm Constraints:** SGD prefers small $\|\mathbf{W}\|_F$.

Inductive Biases: The “Real” Capacity

Effective Capacity $\neq$ Number of Parameters.

What limits complexity?

- **Norm Constraints:** SGD prefers small $\|\mathbf{W}\|_F$.
- **Architecture:** CNNs (locality), RNNs (recurrence).

Inductive Biases: The “Real” Capacity

Effective Capacity $\neq$ Number of Parameters.

What limits complexity?

- **Norm Constraints:** SGD prefers small $\|\mathbf{W}\|_F$.
- **Architecture:** CNNs (locality), RNNs (recurrence).
- **Symmetry:** Learning invariants (e.g., translation).

Inductive Biases: The “Real” Capacity

Effective Capacity $\neq$ Number of Parameters.

What limits complexity?

- **Norm Constraints:** SGD prefers small $\|\mathbf{W}\|_F$.
- **Architecture:** CNNs (locality), RNNs (recurrence).
- **Symmetry:** Learning invariants (e.g., translation).

Hierarchy

1. Data Augmentation
2. Weight Decay
3. Dropout
4. Implicit Regularization

Table of Contents

- 1 Introduction and Motivation
- 2 Universal Approximation Theory
- 3 Capacity and Complexity
- 4 Optimization Theory and Loss Landscapes
- 5 Generalization Theory
- 6 Modern Theoretical Frameworks**
 - Neural Tangent Kernel (NTK)
 - Mean-Field and Information Theory
- 7 Advanced Topics and Frontiers

Table of Contents

- 1 Introduction and Motivation
- 2 Universal Approximation Theory
- 3 Capacity and Complexity
- 4 Optimization Theory and Loss Landscapes
 - Gradient Descent Dynamics
 - Implicit Regularization
- 5 Generalization Theory
- 6 Modern Theoretical Frameworks**
 - **Neural Tangent Kernel (NTK)**
 - Mean-Field and Information Theory
- 7 Advanced Topics and Frontiers
 - Robustness and Adversarial Examples
 - Theory of Generative Models
- 8 Summary and Conclusion

NTK: The Infinite-Width Limit

What happens when width $m \rightarrow \infty$?

NTK: The Infinite-Width Limit

What happens when width $m \rightarrow \infty$?

Core Discovery (Jacot et al. 2018):

Wide neural networks evolve like **Linear Models** with a fixed kernel.

NTK: The Infinite-Width Limit

What happens when width $m \rightarrow \infty$?

Core Discovery (Jacot et al. 2018):

Wide neural networks evolve like **Linear Models** with a fixed kernel.

Training Dynamics:

$$f(\mathbf{x}, \theta_{t+1}) \approx f(\mathbf{x}, \theta_t) - \eta \nabla_{\theta} f(\mathbf{x}, \theta_0)^{\top} \nabla_{\theta} f(\mathbf{x}_{\text{data}}, \theta_0) (f(\mathbf{x}_{\text{data}}, \theta_t) - y) \quad (10)$$

NTK: The Infinite-Width Limit

What happens when width $m \rightarrow \infty$?

Core Discovery (Jacot et al. 2018):

Wide neural networks evolve like **Linear Models** with a fixed kernel.

Training Dynamics:

$$f(\mathbf{x}, \theta_{t+1}) \approx f(\mathbf{x}, \theta_t) - \eta \nabla_{\theta} f(\mathbf{x}, \theta_0)^{\top} \nabla_{\theta} f(\mathbf{x}_{\text{data}}, \theta_0) (f(\mathbf{x}_{\text{data}}, \theta_t) - y) \quad (10)$$

The NTK

$$\Theta(\mathbf{x}_i, \mathbf{x}_j) = \nabla_{\theta} f(\mathbf{x}_i, \theta_0)^{\top} \nabla_{\theta} f(\mathbf{x}_j, \theta_0)$$

Pros and Cons of the NTK View

Pros

- Explains global convergence.
- Rigorous proofs for generalization.
- Connects DL to Kernel Methods.

Cons (The “Lazy” Regime)

- Weights move very little.
- **No Feature Learning:** Kernels are fixed.
- Less powerful than finite-width networks in practice.

Table of Contents

- 1 Introduction and Motivation
- 2 Universal Approximation Theory
- 3 Capacity and Complexity
- 4 Optimization Theory and Loss Landscapes
 - Gradient Descent Dynamics
 - Implicit Regularization
- 5 Generalization Theory
- 6 Modern Theoretical Frameworks**
 - Neural Tangent Kernel (NTK)
 - Mean-Field and Information Theory**
- 7 Advanced Topics and Frontiers
 - Robustness and Adversarial Examples
 - Theory of Generative Models
- 8 Summary and Conclusion

Mean-Field Theory

- **Beyond Lazy Training:** Allows neurons to move significantly.

Mean-Field Theory

- **Beyond Lazy Training:** Allows neurons to move significantly.
- **Particle View:** Treat neurons as particles in a probability distribution $\rho(\theta, t)$.

Mean-Field Theory

- **Beyond Lazy Training:** Allows neurons to move significantly.
- **Particle View:** Treat neurons as particles in a probability distribution $\rho(\theta, t)$.
- **Continuity Equation:** Training described by a PDE:

$$\frac{\partial \rho}{\partial t} = \dots \quad (11)$$

Mean-Field Theory

- **Beyond Lazy Training:** Allows neurons to move significantly.
- **Particle View:** Treat neurons as particles in a probability distribution $\rho(\theta, t)$.
- **Continuity Equation:** Training described by a PDE:

$$\frac{\partial \rho}{\partial t} = \dots \quad (11)$$

Advantage

Captures the **Rich Regime**
where networks actively learn a shared representation (feature learning).

Comparing Training Regimes: “Lazy” vs. “Rich”

Different ways for wide networks to learn.

Feature	Lazy Regime (NTK)	Rich Regime (Mean-Field)
Weight Movement	$\mathcal{O}(1/\sqrt{m})$	$\mathcal{O}(1)$
Kernels	Fixed	Evolve (Learned)
Task Complexity	Linear Modeling	Feature Learning
Initialization	Large scaling	Specific “Rich” scaling
Theory Tool	Linear Algebra / Kernels	PDEs / Optimal Transport

Comparing Training Regimes: “Lazy” vs. “Rich”

Different ways for wide networks to learn.

Feature	Lazy Regime (NTK)	Rich Regime (Mean-Field)
Weight Movement	$\mathcal{O}(1/\sqrt{m})$	$\mathcal{O}(1)$
Kernels	Fixed	Evolve (Learned)
Task Complexity	Linear Modeling	Feature Learning
Initialization	Large scaling	Specific “Rich” scaling
Theory Tool	Linear Algebra / Kernels	PDEs / Optimal Transport

Most “interesting” deep learning happens in the **Rich Regime**, where the model actively changes its internal representation based on the data.

Table of Contents

- 1 Introduction and Motivation
- 2 Universal Approximation Theory
- 3 Capacity and Complexity
- 4 Optimization Theory and Loss Landscapes
- 5 Generalization Theory
- 6 Modern Theoretical Frameworks
- 7 Advanced Topics and Frontiers**
 - Robustness and Adversarial Examples
 - Theory of Generative Models

Table of Contents

- 1 Introduction and Motivation
- 2 Universal Approximation Theory
- 3 Capacity and Complexity
- 4 Optimization Theory and Loss Landscapes
 - Gradient Descent Dynamics
 - Implicit Regularization
- 5 Generalization Theory
- 6 Modern Theoretical Frameworks
 - Neural Tangent Kernel (NTK)
 - Mean-Field and Information Theory
- 7 Advanced Topics and Frontiers**
 - Robustness and Adversarial Examples**
 - Theory of Generative Models
- 8 Summary and Conclusion

Adversarial Examples

Small perturbations, huge consequences.

Adversarial Examples

Small perturbations, huge consequences.

The Phenomenon (Szegedy et al. 2014):

Adding an “imperceptible” noise δ to an image $\mathbf{x}$ can completely change the model’s prediction.

$$f(\mathbf{x} + \delta) \neq f(\mathbf{x}), \quad \|\delta\| \leq \epsilon \quad (12)$$

Adversarial Examples

Small perturbations, huge consequences.

The Phenomenon (Szegedy et al. 2014):

Adding an “imperceptible” noise δ to an image $\mathbf{x}$ can completely change the model’s prediction.

$$f(\mathbf{x} + \delta) \neq f(\mathbf{x}), \quad \|\delta\| \leq \epsilon \quad (12)$$

Why does this happen?

Adversarial Examples

Small perturbations, huge consequences.

The Phenomenon (Szegedy et al. 2014):

Adding an “imperceptible” noise δ to an image $\mathbf{x}$ can completely change the model’s prediction.

$$f(\mathbf{x} + \delta) \neq f(\mathbf{x}), \quad \|\delta\| \leq \epsilon \quad (12)$$

Why does this happen?

- **Linearity:** In high dimensions, small changes in many directions add up.

Adversarial Examples

Small perturbations, huge consequences.

The Phenomenon (Szegedy et al. 2014):

Adding an “imperceptible” noise δ to an image $\mathbf{x}$ can completely change the model’s prediction.

$$f(\mathbf{x} + \delta) \neq f(\mathbf{x}), \quad \|\delta\| \leq \epsilon \quad (12)$$

Why does this happen?

- **Linearity:** In high dimensions, small changes in many directions add up.
- **Curvature:** Decision boundaries are very close to data points.

Robustness vs. Accuracy

Is there a fundamental trade-off?

Robustness vs. Accuracy

Is there a fundamental trade-off?

Robustness vs. Accuracy

Is there a fundamental trade-off?

- **Adversarial Training:** $\min_{\theta} \mathbb{E} \left[\max_{\|\delta\| \leq \epsilon} \mathcal{L}(f(\mathbf{x} + \delta; \theta), y) \right].$

Robustness vs. Accuracy

Is there a fundamental trade-off?

- **Adversarial Training:** $\min_{\theta} \mathbb{E} \left[\max_{\|\delta\| \leq \epsilon} \mathcal{L}(f(\mathbf{x} + \delta; \theta), y) \right]$.
- **The Cost:** Robust models often have lower accuracy on clean data.

Robustness vs. Accuracy

Is there a fundamental trade-off?

- **Adversarial Training:** $\min_{\theta} \mathbb{E} \left[\max_{\|\delta\| \leq \epsilon} \mathcal{L}(f(\mathbf{x} + \delta; \theta), y) \right]$.
- **The Cost:** Robust models often have lower accuracy on clean data.
- **Reason:** Features that are useful for accuracy might be “unrobust” (easy to perturb).

Robustness vs. Accuracy

Is there a fundamental trade-off?

- **Adversarial Training:** $\min_{\theta} \mathbb{E} \left[\max_{\|\delta\| \leq \epsilon} \mathcal{L}(f(\mathbf{x} + \delta; \theta), y) \right]$.
- **The Cost:** Robust models often have lower accuracy on clean data.
- **Reason:** Features that are useful for accuracy might be “unrobust” (easy to perturb).

Theoretical View

Robustness requires more data and higher model capacity than standard training.

Table of Contents

- 1 Introduction and Motivation
- 2 Universal Approximation Theory
- 3 Capacity and Complexity
- 4 Optimization Theory and Loss Landscapes
 - Gradient Descent Dynamics
 - Implicit Regularization
- 5 Generalization Theory
- 6 Modern Theoretical Frameworks
 - Neural Tangent Kernel (NTK)
 - Mean-Field and Information Theory
- 7 Advanced Topics and Frontiers**
 - Robustness and Adversarial Examples
 - Theory of Generative Models**
- 8 Summary and Conclusion

Score-Based and Diffusion Models

From Likelihood to Scores.

Score-Based and Diffusion Models

From Likelihood to Scores.

Score-Based and Diffusion Models

From Likelihood to Scores.

- **Score Function:** $\nabla_{\mathbf{x}} \log p(\mathbf{x})$. It tells us “where the data is”.

Score-Based and Diffusion Models

From Likelihood to Scores.

- **Score Function:** $\nabla_{\mathbf{x}} \log p(\mathbf{x})$. It tells us “where the data is”.
- **Diffusion Process:** Gradually add noise to data until it becomes Gaussian.

Score-Based and Diffusion Models

From Likelihood to Scores.

- **Score Function:** $\nabla_{\mathbf{x}} \log p(\mathbf{x})$. It tells us “where the data is”.
- **Diffusion Process:** Gradually add noise to data until it becomes Gaussian.
- **Reverse Process:** Learn to “denoise” (estimate the score).

Score-Based and Diffusion Models

From Likelihood to Scores.

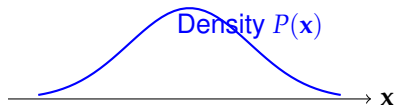
- **Score Function:** $\nabla_{\mathbf{x}} \log p(\mathbf{x})$. It tells us “where the data is”.
- **Diffusion Process:** Gradually add noise to data until it becomes Gaussian.
- **Reverse Process:** Learn to “denoise” (estimate the score).

Key Theoretical Connection

Denoising is equivalent to Score Matching. The model learns the gradient of the data density.

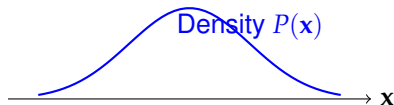
Score Matching: The Engine of Diffusion

- **Question:** How to sample from $P_{data}(\mathbf{x})$?



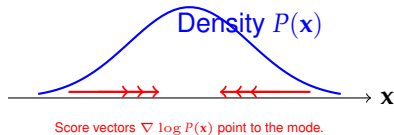
Score Matching: The Engine of Diffusion

- **Question:** How to sample from $P_{data}(\mathbf{x})$?
- **Answer:** Follow the “hill” of the density! $\nabla_{\mathbf{x}} \log P_{data}(\mathbf{x})$.



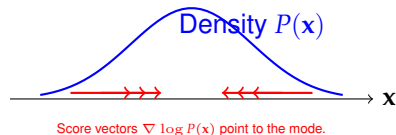
Score Matching: The Engine of Diffusion

- **Question:** How to sample from $P_{data}(\mathbf{x})$?
- **Answer:** Follow the “hill” of the density! $\nabla_{\mathbf{x}} \log P_{data}(\mathbf{x})$.



Score Matching: The Engine of Diffusion

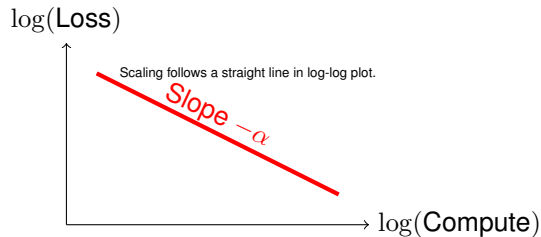
- **Question:** How to sample from $P_{data}(\mathbf{x})$?
- **Answer:** Follow the “hill” of the density! $\nabla_{\mathbf{x}} \log P_{data}(\mathbf{x})$.



By learning to reverse noise,
a neural network is essentially learning to **map the landscape** of the data distribution.

Scaling Laws: Why Scale Matters

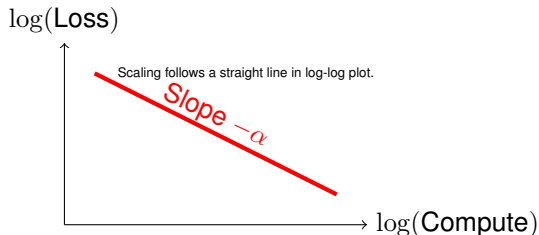
“More is different.” — Phil Anderson



Scaling Laws: Why Scale Matters

“More is different.” — Phil Anderson

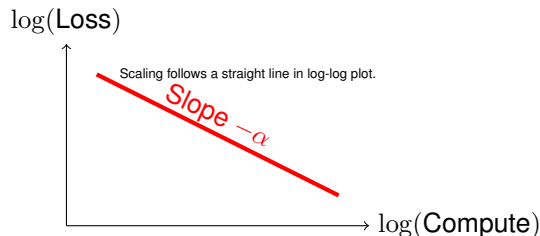
- **Chinchilla Scaling:** N (Params) and D (Data) should grow together.



Scaling Laws: Why Scale Matters

“More is different.” — Phil Anderson

- **Chinchilla Scaling:** N (Params) and D (Data) should grow together.
- **Predictability:** We can predict GPT-4's performance from a model $10,000\times$ smaller!



Future Frontiers of Deep Learning Theory

- **Mechanistic Interpretability:** Moving from understanding “why it generalizes” to “what it’s actually doing” (circuits, induction heads).

Future Frontiers of Deep Learning Theory

- **Mechanistic Interpretability:** Moving from understanding “why it generalizes” to “what it’s actually doing” (circuits, induction heads).
- **Theory of LLMs:** Why do they reason? What is the limit of scaling?

Future Frontiers of Deep Learning Theory

- **Mechanistic Interpretability:** Moving from understanding “why it generalizes” to “what it’s actually doing” (circuits, induction heads).
- **Theory of LLMs:** Why do they reason? What is the limit of scaling?
- **Reasoning & Planning:** Towards a theory of system-2 thinking in NNs.

Future Frontiers of Deep Learning Theory

- **Mechanistic Interpretability:** Moving from understanding “why it generalizes” to “what it’s actually doing” (circuits, induction heads).
- **Theory of LLMs:** Why do they reason? What is the limit of scaling?
- **Reasoning & Planning:** Towards a theory of system-2 thinking in NNs.
- **Safety & Alignment:** Using theory to guarantee model behavior.

Future Frontiers of Deep Learning Theory

- **Mechanistic Interpretability:** Moving from understanding “why it generalizes” to “what it’s actually doing” (circuits, induction heads).
- **Theory of LLMs:** Why do they reason? What is the limit of scaling?
- **Reasoning & Planning:** Towards a theory of system-2 thinking in NNs.
- **Safety & Alignment:** Using theory to guarantee model behavior.

The field is shifting from “foundations” to “mechanisms” and “safety”.

A Global Map of the Theoretical Landscape

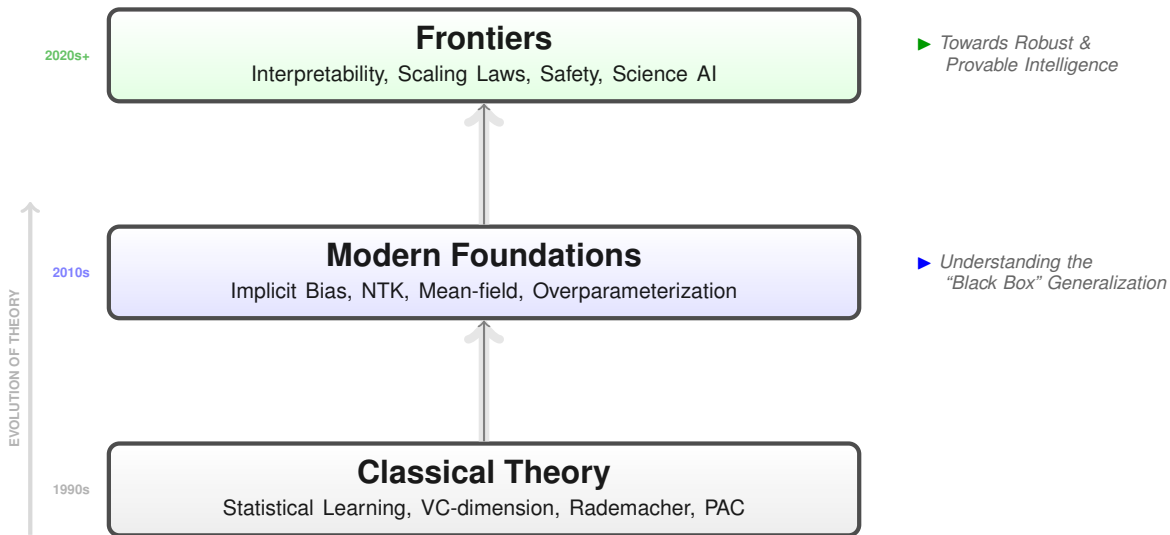


Table of Contents

- 1 Introduction and Motivation
- 2 Universal Approximation Theory
- 3 Capacity and Complexity
- 4 Optimization Theory and Loss Landscapes
- 5 Generalization Theory
- 6 Modern Theoretical Frameworks
- 7 Advanced Topics and Frontiers
- 8 Summary and Conclusion**

The Big Picture

The Big Picture

- 1 **Approximation:** Neural networks can represent anything complex efficiently (depth is better).

The Big Picture

- 1 **Approximation:** Neural networks can represent anything complex efficiently (depth is better).
- 2 **Optimization:** Landscape is messy, but overparameterization and SGD make it navigable.

The Big Picture

- 1 **Approximation:** Neural networks can represent anything complex efficiently (depth is better).
- 2 **Optimization:** Landscape is messy, but overparameterization and SGD make it navigable.
- 3 **Generalization:** We don't overfit because of implicit biases and architecture.

The Big Picture

- 1 **Approximation:** Neural networks can represent anything complex efficiently (depth is better).
- 2 **Optimization:** Landscape is messy, but overparameterization and SGD make it navigable.
- 3 **Generalization:** We don't overfit because of implicit biases and architecture.
- 4 **Frontiers:** Theory is still catching up to LLMs and generative models.

Conclusion

- **Theory explains Practice:** Why we use deep networks and SGD.

Conclusion

- **Theory explains Practice:** Why we use deep networks and SGD.
- **Practice motivates Theory:** The “Generalization Puzzle” changed how we think about complexity.

Conclusion

- **Theory explains Practice:** Why we use deep networks and SGD.
- **Practice motivates Theory:** The “Generalization Puzzle” changed how we think about complexity.
- **Rigorous Intuition:** Even if we can’t prove everything yet, theoretical frameworks (NTK, Mean-Field) help guide architecture design.

Conclusion

- **Theory explains Practice:** Why we use deep networks and SGD.
- **Practice motivates Theory:** The “Generalization Puzzle” changed how we think about complexity.
- **Rigorous Intuition:** Even if we can’t prove everything yet, theoretical frameworks (NTK, Mean-Field) help guide architecture design.

Thank you!